

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №5.

Тема: «Наследование. Виртуальные функции. Полиморфизм»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 13

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

1. Определить абстрактный класс (класс с хотя бы одной чисто виртуальной функцией).
2. Определить иерархию классов, в основе которой будет находиться абстрактный класс.
3. Определить класс Вектор, элементами которого будут указатели на объекты иерархии классов (для демонстрации полиморфизма).
4. Перегрузить для класса Вектор операцию вывода объектов с помощью потоков (`operator<<`).
5. В основной функции продемонстрировать:
 - a. Создание объектов иерархии классов
 - b. Работу класса Вектор
 - c. Перегруженные операции
 - d. Полиморфизм (работа с разными объектами через указатели базового класса)

Базовый класс: ЧЕЛОВЕК (PERSON)

Поля:

- Имя (name) — string
- Возраст (age) — int

Требуется:

- Определить методы изменения полей (сеттеры)
- Сделать класс абстрактным (добавить чисто виртуальную функцию)

Производный класс: EMPLOYEE (СОТРУДНИК)

Наследуется от класса PERSON.

Дополнительные поля:

- Должность (position) — string
- Оклад (salary) — double

Требуется:

- Определить методы изменения полей
- Реализовать вычисление зарплаты сотрудника по формуле:
- Зарплата = Оклад + Премия (% от оклада)

Анализ задачи:

1. Создаем абстрактный базовый класс Person (с чисто виртуальной функцией), производный класс Employee.

2. Создаем класс Vector (или Container), который хранит указатели на Person (Person*), что позволяет хранить в нём как Person, так и Employee.
3. Перегружаем operator << для вывода всех объектов вектора.
4. В main() демонстрируем полиморфизм — когда разные объекты ведут себя по-разному при вызове одной и той же виртуальной функции.

Код:

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// АБСТРАКТНЫЙ БАЗОВЫЙ КЛАСС: PERSON (ЧЕЛОВЕК)
class Person {
protected:
    string name; // Имя человека
    int age;     // Возраст

public:
    // КОНСТРУКТОРЫ
    Person() : name("Неизвестно"), age(0) {
        cout << "Конструктор Person\n";
    }

    Person(string n, int a) : name(n), age(a) {
        cout << "Конструктор Person с параметрами\n";
    }

    // Виртуальный деструктор (обязателен для абстрактных классов)
    virtual ~Person() {
        cout << "Деструктор Person: " << name << "\n";
    }

    // СЕЛЕКТОРЫ И МОДИФИКАТОРЫ
    string getName() const { return name; }
    int getAge() const { return age; }
```

```

void setName(string n) { name = n; }
void setAge(int a) { age = a; }

// ЧИСТО ВИРТУАЛЬНАЯ ФУНКЦИЯ (делает класс абстрактным)
virtual void showInfo() const = 0; // = 0 означает чисто виртуальную функцию

// Виртуальная функция для вычисления зарплаты (по умолчанию 0)
virtual double getSalary() const {
    return 0.0;
}
};

// ПРОИЗВОДНЫЙ КЛАСС: EMPLOYEE (СОТРУДНИК)
class Employee : public Person {
private:
    string position; // Должность
    double salary;   // Оклад

public:
    // КОНСТРУКТОРЫ
    Employee() : Person(), position("Не определена"), salary(0.0) {
        cout << "Конструктор Employee\n";
    }

    Employee(string n, int a, string pos, double sal)
        : Person(n, a), position(pos), salary(sal) {
        cout << "Конструктор Employee с параметрами\n";
    }

    // Деструктор
    ~Employee() {
        cout << "Деструктор Employee: " << name << "\n";
    }

    // СЕЛЕКТОРЫ И МОДИФИКАТОРЫ
    string getPosition() const { return position; }
    double getBaseSalary() const { return salary; }

```

```

void setPosition(string pos) { position = pos; }
void setSalary(double sal) { salary = sal; }

// ПЕРЕОПРЕДЕЛЕНИЕ ВИРТУАЛЬНЫХ ФУНКЦИЙ

// Реализация чисто виртуальной функции (обязательна)
void showInfo() const override {
    cout << "Сотрудник: " << name
        << ", Возраст: " << age
        << ", Должность: " << position
        << ", Оклад: " << salary;
}

// Переопределение функции вычисления зарплаты
double getSalary() const override {
    return salary; // Для сотрудника это просто оклад
}

// ВЫЧИСЛЕНИЕ ЗАРПЛАТЫ С ПРЕМИЕЙ
double calculateTotalSalary(double bonusPercent) const {
    double bonus = salary * bonusPercent / 100.0;
    return salary + bonus;
}
};

// КЛАСС ВЕКТОР (хранит указатели на Person)
class Vector {
private:
    vector<Person*> elements; // Вектор указателей на базовый класс

public:
    // Конструктор по умолчанию
    Vector() {}

    // Деструктор (освобождает память)
    ~Vector() {
        for (Person* p : elements) {

```

```

        delete p; // Удаляем каждый объект
    }
    elements.clear();
}

// ДОБАВЛЕНИЕ ОБЪЕКТОВ В ВЕКТОР
void add(Person* p) {
    elements.push_back(p);
}

// Получение размера вектора
size_t size() const {
    return elements.size();
}

// Доступ к элементу по индексу
Person* get(size_t index) const {
    if (index < elements.size())
        return elements[index];
    return nullptr;
}

// ПЕРЕГРУЗКА ОПЕРАТОРА ВЫВОДА (friend функция)
friend ostream& operator<<(ostream& out, const Vector& vec);
};

// РЕАЛИЗАЦИЯ ОПЕРАТОРА ВЫВОДА ДЛЯ ВЕКТОРА
ostream& operator<<(ostream& out, const Vector& vec) {
    out << "Содержимое вектора (" << vec.elements.size() << " объектов)\n";

    for (size_t i = 0; i < vec.elements.size(); i++) {
        out << "[" << i << "] ";

        // Полиморфизм: вызываем виртуальную функцию showInfo()
        // Фактический тип объекта определится во время выполнения
        vec.elements[i]->showInfo();

        // Если это Employee, выводим зарплату

```

```

        Employee* emp = dynamic_cast<Employee*>(vec.elements[i]);
        if (emp) {
            out << ", Зарплата: " << emp->getSalary();
        }

        out << "\n";
    }

    return out;
}

int main() {
    setlocale(LC_ALL, "ru");

    cout << "1. СОЗДАНИЕ ОБЪЕКТОВ\n\n";

    // Создаём вектор для хранения указателей на Person
    Vector vec;

    // Добавляем объекты разных типов
    // Полиморфизм: все объекты хранятся как Person*,
    // но фактически являются Employee
    vec.add(new Employee("Иван", 25, "Программист", 80000));
    vec.add(new Employee("Мария", 30, "Менеджер", 65000));
    vec.add(new Employee("Петр", 35, "Директор", 120000));

    cout << "\n2. ВЫВОД ВЕКТОРА (ПОЛИМОРФИЗМ)\n\n";

    // Используем перегруженный оператор вывода
    cout << vec;

    cout << "\n3. ДОСТУП К ОТДЕЛЬНЫМ ОБЪЕКТАМ\n\n";

    // Получаем указатель на объект
    Person* p = vec.get(0);
    if (p) {
        cout << "Первый объект:\n";
        p->showInfo(); // Виртуальный вызов
    }
}

```

```

        cout << "\n\n";
    }

    cout << "4. ВЫЧИСЛЕНИЕ ЗАРПЛАТЫ С ПРЕМИЕЙ\n\n";

    // Демонстрируем работу с Employee через указатель на Person
    for (size_t i = 0; i < vec.size(); i++) {
        Employee* emp = dynamic_cast<Employee*>(vec.get(i));
        if (emp) {
            cout << emp->getName() << ": ";
            cout << "Оклад = " << emp->getBaseSalary();
            cout << ", С премией 15% = " << emp->calculateTotalSalary(15.0);
            cout << "\n";
        }
    }

    cout << "\n5. ИЗМЕНЕНИЕ ДАННЫХ\n\n";

    // Изменяем данные через сеттеры
    Employee* emp = dynamic_cast<Employee*>(vec.get(1));
    if (emp) {
        cout << "До изменения: ";
        emp->showInfo();
        cout << "\n";

        emp->setPosition("Старший менеджер");
        emp->setSalary(75000);

        cout << "После изменения: ";
        emp->showInfo();
        cout << "\n";
    }

    cout << "\n6. ИТОГОВЫЙ ВЫВОД\n\n";
    cout << vec;

    cout << "автоматический вызов деструкторов:\n\n";

```



```

return 0;

// Деструктор Vector автоматически удалит все объекты
}

```

Результат выполнения:

```

Консоль отладки Microsoft Visual Studio
1. СОЗДАНИЕ ОБЪЕКТОВ

Конструктор Person с параметрами
Конструктор Employee с параметрами
Конструктор Person с параметрами
Конструктор Employee с параметрами
Конструктор Person с параметрами
Конструктор Employee с параметрами

2. ВЫВОД ВЕКТОРА (ПОЛИМОРФИЗМ)

Содержимое вектора (3 объектов)
[0] Сотрудник: Иван, Возраст: 25, Должность: Программист, Оклад: 80000, Зарплата: 80000
[1] Сотрудник: Мария, Возраст: 30, Должность: Менеджер, Оклад: 65000, Зарплата: 65000
[2] Сотрудник: Петр, Возраст: 35, Должность: Директор, Оклад: 120000, Зарплата: 120000

3. ДОСТУП К ОТДЕЛЬНЫМ ОБЪЕКТАМ

Первый объект:
Сотрудник: Иван, Возраст: 25, Должность: Программист, Оклад: 80000

4. ВЫЧИСЛЕНИЕ ЗАРПЛАТЫ С ПРЕМИЕЙ

Иван: Оклад = 80000, С премией 15% = 92000
Мария: Оклад = 65000, С премией 15% = 74750
Петр: Оклад = 120000, С премией 15% = 138000

5. ИЗМЕНЕНИЕ ДАННЫХ

До изменения: Сотрудник: Мария, Возраст: 30, Должность: Менеджер, Оклад: 65000
После изменения: Сотрудник: Мария, Возраст: 30, Должность: Старший менеджер, Оклад: 75000

6. ИТОГОВЫЙ ВЫВОД

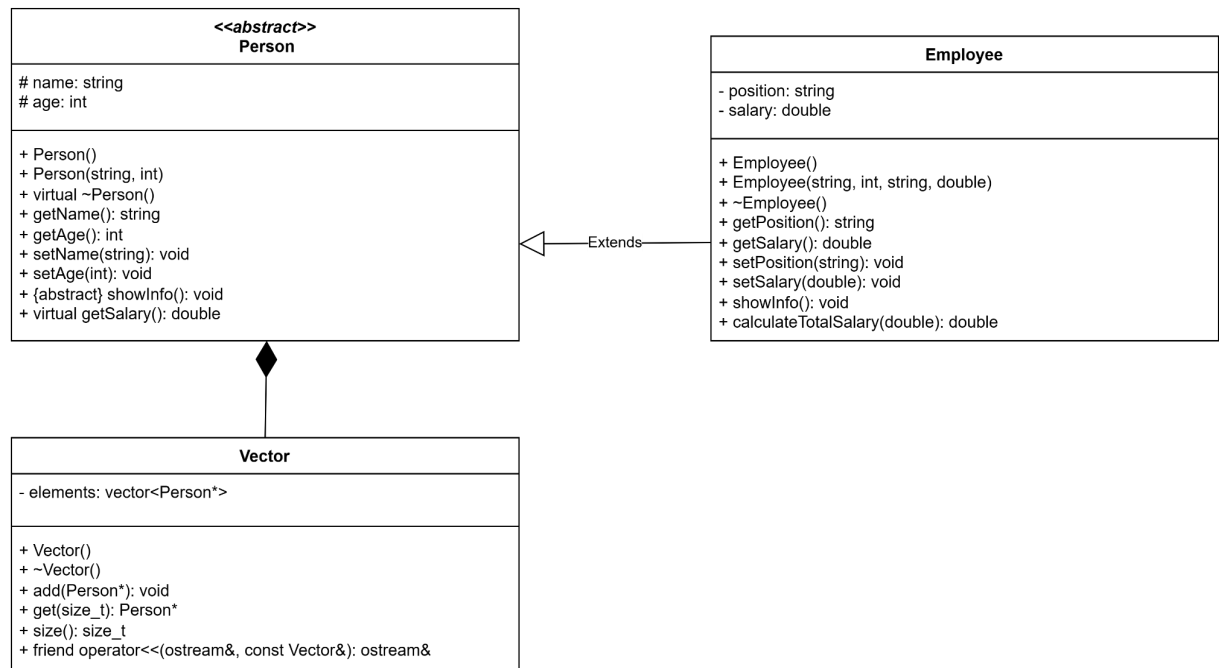
Содержимое вектора (3 объектов)
[0] Сотрудник: Иван, Возраст: 25, Должность: Программист, Оклад: 80000, Зарплата: 80000
[1] Сотрудник: Мария, Возраст: 30, Должность: Старший менеджер, Оклад: 75000, Зарплата: 75000
[2] Сотрудник: Петр, Возраст: 35, Должность: Директор, Оклад: 120000, Зарплата: 120000
автоматический вызов деструкторов:

Деструктор Employee: Иван
Деструктор Person: Иван
Деструктор Employee: Мария
Деструктор Person: Мария
Деструктор Employee: Петр
Деструктор Person: Петр

C:\Users\Elena Lapteva\source\repos\labs\x64\Debug\labs.exe (процесс 5432) завершил работу с кодом 0

```

UML:



Вопросы:

1. Чисто виртуальный метод содержит признак `= 0` вместо тела, например:
`virtual void f(int) = 0;`
2. Класс, содержащий хотя бы один чисто виртуальный метод, называется абстрактным.
3. Абстрактные классы предназначены для представления общих понятий, которые предполагается конкретизировать в производных классах. Абстрактный класс может использоваться только в качестве базового для других классов.
4. Функции, работающие с объектом любого типа в пределах одной иерархии
5. Полиморфизм — это механизм вызова переопределённого метода через указатель на базовый класс (как `p→Show()`). Принцип подстановки — это возможность использовать объекты производных классов (`Car`, `Lorry`) везде, где ожидается объект базового класса (`Object`), что показано в функции `main()` при добавлении объектов в `Vector`.
6. В работе представлена иерархия:
 - **Person** (абстрактный класс с чисто виртуальным методом `showInfo()`)
 - **Employee** (наследует **Person**, переопределяет `showInfo()` и добавляет метод `calculateTotalSalary()`)

Класс Vector агрегирует указатели на Person*, что позволяет хранить в одном векторе объекты разных производных классов и демонстрировать полиморфизм.

7. Позднее связывание (определение вызываемой функции на этапе выполнения программы) используется при вызове виртуальных методов через указатель или ссылку на базовый класс